2

Python for Security Professionals – Beyond the Basics

This chapter looks into complex Python applications designed exclusively for security professionals, pushing you beyond the fundamentals and into the field of cutting-edge cybersecurity practices.

By the end of this chapter, you will not only have a thorough understanding of Python's role in cybersecurity, but you will also have firsthand experience designing a comprehensive security tool. You will obtain the expertise required to effectively address real-world security concerns through practical examples and in-depth explanations.

In this chapter, we are going to cover the following main topics:

- Utilizing essential security libraries
- Harnessing advanced Python techniques for security
- Compiling a Python library
- Advanced Python features

Utilizing essential security libraries

At the heart of network analysis is the practice of network scanning. **Network scanning** is a technique used to discover devices on a network, determine open ports, identify available services, and uncover vulnerabilities. This information is invaluable for security assessments and maintaining the overall security of a network.

Without further ado, let us write a network scanner. **Scapy** will be our library of choice.

Scapy is a powerful packet manipulation tool that allows users to capture, analyze, and forge network packets. It can be used for network discovery, security testing, and forensic analysis.

HOW CAN I DETERMINE THAT SCAPY IS THE PREFERRED LIBRARY FOR OUR TOOL?

You'll need to do some Google searches, and you can also use **https://pypi.org/** *to find modules that suit your needs.*

So, now that we have our library, how can I find the modules from Scapy that are needed for our tool? For that, you can make use of the documentation that is available either at **https://pypi.org/** or the GitHub repository for the library (**https://github.com/secdev/scapy**).

One effective method for network discovery is utilizing an **Address Resolution Protocol** (**ARP**) scan to query devices on the local network, gathering their IP and MAC addresses. With Scapy, a powerful packet manipulation library, we can create a simple yet efficient script to perform this task. Scapy's flexibility allows for detailed customization of packets, enabling precise control over the ARP requests we send out. This not only increases the efficacy of the scanning process but also minimizes any potential network disturbance. The following is an optimized script that demonstrates this process.

The first step in any Python script involves loading the necessary modules.

Import the necessary modules from Scapy, including **`ARP`**, **`Ether`** (**Ethernet frame**), and **`srp`** (**send and receive packets**):

```
1. # Import the necessary modules from Scapy
2. from scapy.all import ARP, Ether, srp
3.
```

We can start by creating the function to perform an ARP scan. The **`arp_scan`** function takes a target IP range as input and creates an ARP request packet for each IP address in the specified range. It sends the packets, receives responses, and extracts IP and MAC addresses, storing them in a list called **`devices_list`**:

```
4. # Function to perform ARP scan
5. def arp_scan(target_ip):
6.     # Create an ARP request packet
7.     arp_request = ARP(pdst=target_ip)
8.     # Create an Ethernet frame to encapsulate the ARP request
9.     ether_frame = Ether(dst="ff:ff:ff:ff:ff:ff")  # Broadcasting to all devices in the netwo
10.
11.     # Combine the Ethernet frame and ARP request packet
12.     arp_request_packet = ether_frame / arp_request
13.
14.     # Send the packet and receive the response
15.     result = srp(arp_request_packet, timeout=3, verbose=False)[0]
16.
17.     # List to store the discovered devices
18.     devices_list = []
19.
20.     # Parse the response and extract IP and MAC addresses
21.     for sent, received in result:
22.         devices_list.append({'ip': received.psrc, 'mac': received.hwsrc})
23.
24.     return devices_list
25.
```

Another task is displaying the scan result in the input-output stream. The **`print_scan_results`** function takes the **`devices_list`** as input and prints the discovered IP and MAC addresses in a formatted manner:

```
26. # Function to print scan results
27. def print_scan_results(devices_list):
28.     print("IP Address\t\tMAC Address")
29.     print("-----------------------------------------")
30.     for device in devices_list:
31.         print(f"{device['ip']}\t\t{device['mac']}")
32.
```

The **`main`** function, the primary function to initiate the script, takes the target IP range as input, initiates the ARP scan, and prints the scan results:

```
33. # Main function to perform the scan
34. def main(target_ip):
35.     print(f"Scanning {target_ip}...")
36.     devices_list = arp_scan(target_ip)
37.     print_scan_results(devices_list)
38.
```

The **`if __name__ == "__main__":`** block makes the script executable in a command line and capable of accepting parameters. The script prompts the user to enter the target IP range, such as **`192.168.1.1/24`**:

```
39. # Entry point of the script
40. if __name__ == "__main__":
41.     # Define the target IP range (e.g., "192.168.1.1/24")
42.     target_ip = input("Enter the target IP range (e.g., 192.168.1.1/24): ")
43.     main(target_ip)
```

To run the script, ensure you are in the virtual environment, have the Scapy library installed (**`pip install scapy`**), and execute it with elevated privilege. It will perform an ARP scan on the specified IP range and print the discovered devices' IP and MAC addresses.

The result should look like this:

```
sudo python3 arp-scanner.py
Password:
WARNING: No IPv4 address found on awdl0 !
WARNING: No IPv4 address found on llw0 !
WARNING: more No IPv4 address found on en1 !
Enter the target IP range (e.g., 192.168.1.1/24): 192.168.29.1/24
Scanning 192.168.29.1/24...
IP Address              MAC Address
-----------------------------------------
192.168.29.1            b4:a7:c6:fa:df:c8
192.168.29.2            46:12:a7:5a:83:1e
192.168.29.242          82:c3:11:78:54:d4
```

Figure 2.1 – Output of the executed ARP scanner script

The preceding is a simple example of how powerful Python libraries are. In addition to Scapy, I have listed some of the most common libraries that we can utilize in our security operations in the previous chapter. The

given libraries are just a few; there are many more that you can utilize for different needs.

Let us now deep dive into harnessing advanced Python techniques, utilizing our newfound knowledge to navigate complex security terrains with utmost precision, innovation, and resilience.

Harnessing advanced Python techniques for security

Using the IP addresses obtained from our network scan, we will create a Python module, import it into our program, and perform a port scan on these IPs. Along the way, we will also cover some advanced Python concepts.

In this section, we will delve into the intricacies of complex Python concepts, starting with basic elements such as **object-oriented programming** and **list comprehensions**. We will break down these advanced aspects of Python, making them easy to understand, and discuss how they can be utilized in the realm of technical security testing.

Before starting the port scanner, we need to import the necessary modules in Python, in this case, the **`socket`** module. The **`socket`** module is a key tool used to create networking interfaces in Python. It is important to do so because the **`socket`** module provides us with the ability to connect with other computers over a network. This connection with other computers is a crucial starting point for any port scanning procedure. Hence, importing the **`socket`** module is the first step in setting up the port scanner.

In this initial stage, we will be importing the necessary modules as usual:

```
1. import socket
2. import threading
3. import time
```

We will initiate the creation of a **`PortScanner`** class. In the realm of Python, a class serves as a blueprint for creating objects that encapsulate a set of variables and functions:

```
5. #Class Definition
6. class PortScanner:
```

The first method that one encounters within a class is the **`__init__`** method. This method is crucial as it is used to initialize the instances of a class or to set the default values. The **`__init__`** method is a special method in Python classes. It acts as a constructor and is automatically called when a new instance of the class is created. In this method:

- **`self`**: This refers to the instance of the class being created. It allows you to access the instance's attributes and methods.

- **`target_host`**: A parameter representing the target host (e.g., a website or IP address) that the port scanner will scan.
- **`start_port`** and **`end_port`** are parameters representing the range of ports to be scanned, from **`start_port`** to **`end_port`**.
- **`open_ports`**: An empty list that will store the list of open ports found during the scanning process. This list is specific to each instance of the **`PortScanner`** class:

```
    def __init__(self, target_host, start_port, end_port):
        self.target_host = target_host
        self.start_port = start_port
        self.end_port = end_port
        self.open_ports = []
```

The method named **`is_port_open`** is thoroughly discussed next. This procedure is designed to check if certain network ports are open, which is crucial for identifying potential vulnerabilities in a network's security system.

The following are the parameters for the method:

- **`self`**: This represents the instance of the class on which the method is called.
- **`port`**: This is a parameter representing the port number to be checked for openness.

```
#is_port_open Method
    def is_port_open(self, port):
        try:
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
                s.settimeout(1)
                s.connect((self.target_host, port))
            return True
        except (socket.timeout, ConnectionRefusedError):
            return False
```

The following are the principles outlined in the preceding code block elaborated on in detail:

- **`with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:`**: This line creates a new socket object:
  - **`socket.AF_INET`**: This indicates that the socket will use IPv4 addressing.
  - **`socket.SOCK_STREAM`**: This indicates that the socket is a TCP socket, used for streaming data.
- **`s.settimeout(1)`**: This sets a timeout of **`1`** second for the socket. connection attempt. If the connection does not succeed within **`1`** second, a **`socket.timeout`** exception will be raised.
- **`s.connect((self.target_host, port))`**: This attempts to establish a connection to the target host (**`self.target_host`**) on the specified port (**`port`**).
- **Handling a connection result**: If the connection attempt is successful (i.e., the port is open and accepts connections), the code inside the **`try`**

block is executed without errors. In this case, the method immediately returns **`True`**, indicating that the port is open.

- **Handling exceptions**: If the connection attempt results in a timeout (**`socket.timeout`**) or **`ConnectionRefusedError`**, it means the port is closed or unreachable. These exceptions are caught in the **`except`** block, and the method returns **`False`**, indicating that the port is closed.

The **`is_port_open`** method is used internally within the **`PortScanner`** class. It provides a simple and reusable way to check whether a specific port on the target host is open or closed. By encapsulating this logic into a separate method, the code becomes more modular and easier to maintain. The method allows the **`PortScanner`** class to efficiently determine the status of ports during the scanning process.

Next, we discuss the **`scan_ports`** method. This method is an intricate process that systematically scans and analyzes network ports.

The following is the parameter for the method (**`self`**); it represents the instance of the class on which the method is called:

```
21. #scan_ports Method
22.     def scan_ports(self):
23.         open_ports = [port for port in range(self.start_port, self.end_port + 1) if self.is_
24.         return open_ports
```

The following are the principles outlined in the preceding code block elaborated on in detail:

- **`[port for port in range(self.start_port, self.end_port + 1) if self.is_port_open(port)]`**: This is a list comprehension that iterates over each port in the specified range (**`self.start_port`** to **`self.end_port`**). For each port, it checks whether the port is open using the **`is_port_open`** method. If the port is open, it is included in the list of **`open_ports`**.
  Comprehensions in Python are concise and expressive ways to create lists, sets, dictionaries, and generators. They allow you to create these data structures by applying an expression to each item in **`iterable`** and optionally filtering the items based on a condition.
  The syntax for the list comprehension will be as follows:

  ```
  1. new_list = [expression for item in iterable if condition]
  ```

  The constituents of the structure can be elucidated as follows:
  - **`expression`**: The operation to perform on each **`item`**.
  - **`item`**: A variable representing an element from **`iterable`**.
  - **`iterable`**: A sequence of elements (e.g., list, range, string) that the comprehension iterates over.
  - **`condition`**: This is optional and represents an optional filter to include items in the new list based on a certain condition.

  Referring to the **`[port for port in range(self.start_port, self.end_port + 1) if self.is_port_open(port)]`** line in the

`scan_ports` method, it is a list comprehension. Its components can be dissected and expounded upon as follows:

- `port`: This represents each port number in the range from `self.start_port` to `self.end_port`.
- `range(self.start_port, self.end_port + 1)`: This generates a sequence of port numbers from `self.start_port` to `self.end_port`.
- `if self.is_port_open(port)`: This acts as a filter. Only ports for which `self.is_port_open(port)` returns `True` are included in the new list.

The procedure of Implementation will be as follows:

1. **Iteration over range**: The list comprehension iterates over each port in the specified range: `(self.start_port to self.end_port)`.
2. **Filtering with the if condition**: For each port, the `if self.is_port_open(port)` condition is checked. If the port is open (`is_port_open` returns `True`), the port is included in the list.
3. **Expression**: The port itself is the expression. This means that the list will contain the port numbers of all open ports within the specified range.
4. **Result**: The `open_ports` list contains all open ports between `self.start_port` and `self.end_port`.

The benefits of list comprehensions can be summarized as follows:

- **Readability**: List comprehensions are concise and readable, making the code more understandable.
- **Compactness**: They allow you to achieve the same functionality with fewer lines of code compared to traditional loops.
- **Clarity**: They express the intent of creating a new list from an existing `iterable`, improving code clarity.

In summary, list comprehensions provide an elegant and Pythonic way to create lists by transforming and filtering elements from `iterable`. They are a fundamental part of Python's expressive syntax, enabling developers to write efficient and readable code.

- `for port in range(self.start_port, self.end_port + 1)`:
  This loop iterates over each port number in the specified range, inclusive of both `start_port` and `end_port`.
- `if self.is_port_open(port)`:
  For each port in the range, it calls the `is_port_open` method to check whether the port is open (using the `is_port_open` helper method explained earlier).
- `open_ports = [port for port in range(self.start_port, self.end_port + 1) if self.is_port_open(port)]`:
  The list comprehension constructs a list of open ports by including only those ports for which `is_port_open(port)` returns `True`. This list comprehensively identifies and captures all open ports within the specified range.
- `return open_ports`: The method returns the list of open ports found during the scan.
  The `scan_ports` method provides a concise way to scan a range of ports and collect the list of open ports. It utilizes list comprehension, a powerful feature in Python, to create the `open_ports` list in a single

line. By using this method, the **PortScanner** class can easily and efficiently identify open ports within a specified range on the target host. This method is integral to the functionality of the **PortScanner** class, as it encapsulates the logic of scanning and identifying open ports.

Now, we will focus on the **main()** function. The following are its features:

- **target_host = input("Enter target host: ")**: This prompts the user to input the target host (e.g., a website domain or IP address) and stores the input in the **target_host** variable.
- **start_port = int(input("Enter starting port: "))**: This prompts the user to input the starting port number and stores the input after converting it to an integer in the **start_port** variable.
- **end_port = int(input("Enter ending port: "))**: This prompts the user to input the ending port number and stores the input after converting it to an integer in the **end_port** variable.

The following is the **main()** function code section:

```
25. def main():
26.     target_host = input("Enter target host: ")
27.     start_port = int(input("Enter starting port: "))
28.     end_port = int(input("Enter ending port: "))
29.
30.     scanner = PortScanner(target_host, start_port, end_port)
31.
32.     open_ports = scanner.scan_ports()
33.     print("Open ports: ", open_ports)
34.
```

The following are the code execution steps explained in depth:

- **scanner = PortScanner(target_host, start_port, end_port)**: This creates an instance of the **PortScanner** class, passing the **target_host**, **start_port**, and **end_port** as parameters. This instance will be used to scan ports.
- **open_ports = scanner.scan_ports()**: This calls the **scan_ports** method of the **PortScanner** instance to scan ports. This method returns a list of open ports using list comprehension, as explained in previous discussions.
- **print("Open ports (using list comprehension):", open_ports)**: This prints the list of open ports obtained from the **scan_ports** method using list comprehension.

Finally, add the following code block to call the **main** method when the file is executed as a script, but not when it is utilized as an import module:

```
35. if __name__ == "__main__":
36.     main()
```

The **if __name__ == "__main__":** line checks whether the script is being run directly (not imported as a module). When you execute the script directly, this condition evaluates to **True**. If the script is being run directly, it

calls the **`main()`** function, initiating the process of input gathering, port scanning, and displaying the results.

In summary, this script prompts the user for a target host, starting port, and ending port. It then creates an instance of the **`PortScanner`** class and uses list comprehension to scan ports within the specified range. The open ports are displayed to the user. The script is structured to be run directly as a standalone program, allowing users to interactively scan ports based on their input.

Now, let us convert our code into a Python library, so we can install and use this library in any of our Python scripts.

Compiling a Python library

Creating a Python library involves packaging your code in a way that allows others to easily install, import, and use it in their projects. To convert your code into a Python library, follow these steps:

1. **Organize your code**: Ensure our code is well-organized and follows the package structure:

   ```
   portscanner/
   |-- portscanner/
   |   |-- __init__.py
   |   |-- portscanner.py
   |-- setup.py
   |-- README.md
   ```

   The components of a Python library are as follows:

   1. **`portscanner/`**: The main folder containing your package
   2. **`portscanner/__init__.py`**: An empty file indicating that **`portscanner`** is a Python package
   3. **`portscanner/scanner.py`**: Your **`PortScanner`** class and related functions
   4. **`setup.py`**: The script for packaging and distributing your library
   5. **`README.md`**: Documentation explaining how to use your library

2. **Update setup.py**:

   ```
   1. from setuptools import setup
   2.
   3. setup(
   4.     name='portscanner',
   5.     version='0.1',
   6.     packages=['portscanner'],
   7.     install_requires=[],
   8.     entry_points={
   9.         'console_scripts': [
   10.             'portscanner = portscanner.portscanner:main'
   11.         ]
   12.     }
   13. )
   ```

   In this file, you specify the package name (**`portscanner`**), version number, and the packages to include (use **`find_packages()`** to automati-

cally discover packages). Add any dependencies your library requires to the `install_requires` list.

3. **Package your code**: In your terminal, navigate to the directory containing your `setup.py` file and run the following command to create a source distribution package:

   ```
   python setup.py sdist
   ```

   This will create a `.tar.gz` file in the `dist` directory.

4. **Publish your library (optional)**: If you want to make your library publicly available, you can publish it on the **Python Package Index (PyPI)**. You will need to create an account on PyPI and install `twine` if you have not already:

   ```
   pip install twine
   ```

   Then, upload your package to PyPI using `twine`:

   ```
   twine upload dist/*
   ```

5. **Install and use your library**: To test your library, you can install it locally using `pip`. In the same directory as your `setup.py`, run the following command:

   ```
   pip install .
   ```

   Now, you can import and use your `PortScanner` class from the `portscanner` package:

   ```
   1. from portscanner.portscanner import PortScanner
   2.
   3. scanner = PortScanner(target_host, start_port, end_port)
   4. open_ports = scanner.scan_ports()
   5. print("Open ports: ", open_ports)
   ```

   Our code is now packaged as a Python library, ready for distribution and use by others.

As we bring our discussion on compiling a Python library to a close, we turn toward the exploration of advanced Python features to further enhance our knowledge and skills in cybersecurity. The following section will delve deeper into these sophisticated components of Python.

Advanced Python features

We can combine our library created in the earlier section into our network mapping tool and transform our code, which does a network scan and a port scan of the discovered IP addresses. Let us keep it there and talk about some more advanced Python features, starting with **decorators**.

Decorators

Decorators are a powerful aspect of Python's syntax, allowing you to modify or enhance the behavior of functions or methods without changing their source code.

For an improved understanding of Python decorators, we will delve into the examination of the following code:

```
1. def timing_decorator(func):
2.     def wrapper(*args, **kwargs):
3.         start_time = time.time()  # Record the start time
4.         result = func(*args, **kwargs)  # Call the original function
5.         end_time = time.time()  # Record the end time
6.         print(f"Scanning took {end_time - start_time:.2f} seconds.")  # Calculate and print t
7.         return result  # Return the result of the original function
8.     return wrapper  # Return the wrapper function
```

The following are the code execution steps explained in depth:

- **`timing_decorator(func)`**: This decorator function, known as **`timing_decorator(func)`**, operates by accepting a function (**`func`**) as a parameter, and then producing a new function (**`wrapper`**). **`wrapper`** fundamentally functions as a housing for the original function, offering an extra layer of operation.
- **`wrapper(*args, **kwargs)`**: This is the **`wrapper`** function returned by the decorator. It captures the arguments and keyword arguments passed to the original function, records the start time, calls the original function, records the end time, calculates the time taken, prints the duration, and finally returns the result of the original function.

Let us examine how to utilize this decorator in the code:

```
1.     @timing_decorator
2.     def scan_ports(self):
3.         open_ports = [port for port in range(self.start_port, self.end_port + 1) if self.is_p
4.         return open_ports
```

Here, **`@timing_decorator`** decorates the **`scan_ports`** method using **`timing_decorator`**. It is equivalent to writing **`scan_ports = timing_decorator(scan_ports)`**.

`timing_decorator` measures the time taken for the **`scan_ports`** method to execute and prints the duration. This is a common use case for decorators, showcasing how they can enhance the functionality of methods in a clean and reusable way. Decorators provide a flexible and elegant approach to modifying or extending the behavior of functions and methods in Python.

Next, we will explore various scenarios where decorators can be effectively employed for enhancing code functionality in an uncomplicated manner. We will highlight the benefits of using decorators in improving code maintainability, readability, and reusability, ultimately contributing to a more secure and optimized code base.

The following are the use cases and benefits of decorators:

- **Code reusability**: Decorators allow you to encapsulate reusable functionality and apply it to multiple functions or methods without dupli-

cating code.

- **Logging and timing**: Decorators are often used for logging, timing, and profiling functions to monitor their behavior and performance.
- **Authentication and authorization**: Decorators can enforce authentication and authorization checks, ensuring that only authorized users can access certain functions or methods.
- **Error handling**: Decorators can handle exceptions raised by functions, providing consistent error handling across different parts of the code.
- **Aspect-oriented programming (AOP)**: Decorators enable AOP, allowing you to separate cross-cutting concerns (e.g., logging, security) from the core logic of functions or methods.
- **Method chaining and modification**: Decorators can modify the output or behavior of functions, enabling method chaining or transforming return values.

This previous code usage appears to be almost complete, however, it appears that our port scanner tool we started at the beginning of this chapter utilizes a list to store the open ports and then return them, which makes the code not so memory efficient, so let's introduce **generators**, another Python concept.

Generators

Generators in Python are special types of functions that return a sequence of results. They allow you to declare a function that behaves like an iterator, iterating over it in a for-loop or explicitly with the next function, while also maintaining the program's internal state and pausing execution between values, thus optimizing memory use.

The characteristics of the generator are detailed and explained more thoroughly as follows:

- **Lazy evaluation**: Generators use lazy evaluation, meaning they produce values on the fly and only when requested. This makes them memory efficient, especially when dealing with large datasets or potentially infinite sequences.
- **Memory efficiency**: Generators do not store all values in memory at once. They generate each value one at a time, consuming minimal memory. This is particularly beneficial for working with large data streams.
- **Efficient iteration**: Generators are `iterable` objects, allowing them to be used in loops and comprehensions just like lists. However, they generate values efficiently, processing each item as it is needed.
- **Infinite sequences**: Generators can represent infinite sequences. Since they produce values on demand, you can create a generator that theoretically generates values forever.

We will enhance our understanding of Python generators by thoroughly analyzing the following code:

```
    def scan_ports_generator(self):
        for port in range(self.start_port, self.end_port + 1):
            if self.is_port_open(port):
                yield port
```

Following are the code execution steps explained in depth to understand the generator function behavior:

- **`for port in range(self.start_port, self.end_port + 1)`**: This loop iterates over each port number in the specified range (**`self.start_port`** to **`self.end_port`**), inclusive of both **`start_port`** and **`end_port`**.
- **`if self.is_port_open(port)`**: For each port in the range, it calls the **`is_port_open`** method to check whether the port is open (using the **`is_port_open`** helper method explained earlier).
- **`yield port`**: If the port is open, the **`yield`** keyword is used to produce a value (**`port`**) from the generator. The **`generator`** function maintains its state, allowing it to resume execution from where it left off when the next value is requested.

The use cases of the generator are detailed and explained further as follows:

- **Processing large files**: Reading large files line-by-line without loading the entire file into memory
- **Stream processing**: Analyzing real-time data streams where data is continuously generated
- **Efficient filtering**: Generating filtered sequences without creating intermediate lists
- **Mathematical calculations**: Generating sequences such as Fibonacci numbers or prime numbers on the fly

Generators are a powerful concept in Python, enabling the creation of memory-efficient and scalable code, especially when working with large datasets or sequences. They are a fundamental feature of Python's expressive syntax and are widely used in various programming scenarios.

Upon utilizing advanced features such as decorators and generators, our code will take on a certain form. This is how the code should appear in its final iteration:

```
import socket
import time

#Class Definition
class PortScanner:
    def __init__(self, target_host, start_port, end_port):
        self.target_host = target_host
        self.start_port = start_port
        self.end_port = end_port
        self.open_ports = []
    #timing_decorator Decorator Method
    def timing_decorator(func):
```

```
        def wrapper(*args, **kwargs):
            start_time = time.time()
            result = func(*args, **kwargs)
            end_time = time.time()
            print(f"Scanning took {end_time - start_time:.2f} seconds.")
            return result
        return wrapper
    #is_port_open Method
    def is_port_open(self, port):
        try:
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
                s.settimeout(1)
                s.connect((self.target_host, port))
            return True
        except (socket.timeout, ConnectionRefusedError):
            return False
    #scan_ports Method
    @timing_decorator
    def scan_ports(self):
        open_ports = [port for port in range(self.start_port, self.end_port + 1) if self.is_
        return open_ports
    #scan_ports_generator Method
    @timing_decorator
    def scan_ports_generator(self):
        for port in range(self.start_port, self.end_port + 1):
            if self.is_port_open(port):
                yield port

def main():
    target_host = input("Enter target host: ")
    start_port = int(input("Enter starting port: "))
    end_port = int(input("Enter ending port: "))

    scanner = PortScanner(target_host, start_port, end_port)

    open_ports = scanner.scan_ports()
    print("Open ports: ", open_ports)

    open_ports_generator = scanner.scan_ports_generator()
    print("Open ports (using generator):", list(open_ports_generator))

if __name__ == "__main__":
    main()
```

Having explored the intricacies of a port scanner script, its modularization, and the use of advanced features such as decorators and generators, our next task is to delve into the subsequent steps. This translates into a hands-on activity, allowing readers an opportunity to further analyze and comprehend coding practices.

Summary

In this chapter, we learned how to use Python to create a network mapper and a port scanner. We covered several complex Python topics along the road, such as object-oriented programming, comprehensions, decorators, generators, and how to package a Python program into a library.

You will be able to build more elegant Python code that stands out from the crowd with these. The presented concepts are difficult to execute at first, but once you get the feel of it, you will never consider coding in any other way.

The next chapter will dive into the exciting and paramount field of web security through the usage of Python. It will explore various strategies, tools, and techniques that Python offers to identify and neutralize security threats, alongside understanding the fundamentals of web security itself. This knowledge is imperative in today's digital age, where security breaches can come at a high cost.

Activity

Now, I will leave it up to you to package both the network scanner and the port scanner into a library and use the libraries to write a more compact script that does a network scan and scan for open ports for each IP that is found.

Readers are recommended to follow the outlined steps to reinforce the concepts presented in this chapter. Every task designed within this activity aims to test and solidify the reader's comprehension effectively:

1. Package both code snippets into the library.
2. Name your new Python file **`network-and-port-scanner.py`**.
3. Import both libraries into the new program.
4. Use argument parsing to obtain the IP and port ranges for scanning.
5. Pass the IP address range to the network mapper for ARP scanning and write the IPs to a file.
6. Read the IPs from the file and provide the discovered IP addresses with the port range to the port scanner.
7. Print the IPs and ports into a table in a more visually appealing manner.